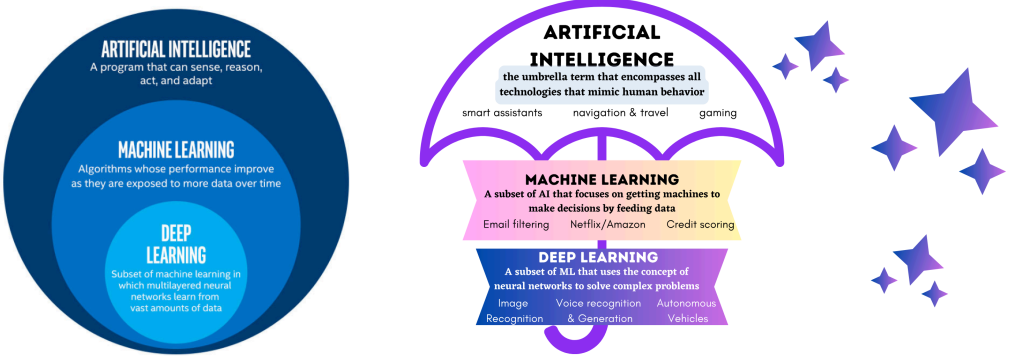
Machine Learning Recap Notes

1. Introduction

What is Machine Learning?



Machine Learning (ML) is a subset of Artificial Intelligence (AI) that enables computers to learn and make decisions from data without being explicitly programmed for every task.

So instead of writing specific instructions, we provide algorithms with examples and let them **find patterns**.

Think of ML like teaching a child to recognize animals. Instead of describing every detail of what makes a cat a cat, you show them hundreds of pictures of cats and dogs. Eventually, they learn to distinguish between them on their own.

Why is Machine Learning Important?

- **Automation**: Automate complex decision-making processes
- **Pattern Recognition**: Find hidden patterns in large datasets
- **Prediction**: Make predictions about future events
- **Personalization**: Customize experiences (like Netflix recommendations)
- **Efficiency**: Process vast amounts of data quickly

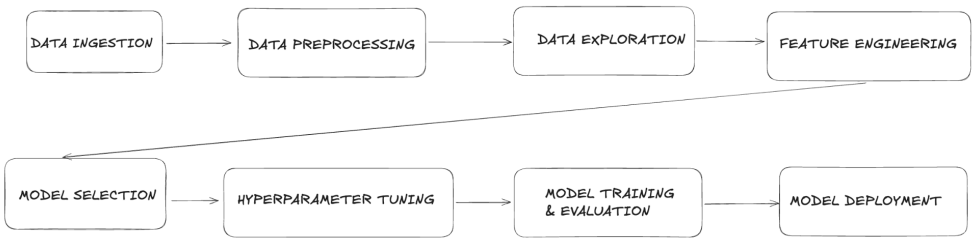
2. Glossary

- **Features (X)**: Input variables (like height, weight, age)

- **Target/Label (y)**: What we want to predict (like price, category)
- **Data**: The information we feed to our algorithms
- **Dataset**: Collection of data
- **DataFrame**: 2D table-like data structure in pandas
- **Training Set**: Dataset used to teach the algorithm

- **Test Set**: Dataset used to evaluate performance
- **Algorithm**: The mathematical method used to find patterns
- **Model**: The result after training an algorithm on data

3. Machine Learning Pipeline



Step	Short Definition
1. Data Ingestion	Load data from files, APIs, or databases
2. Data Preprocessing	Clean and fix messy data
3. Data Exploration	Look at data to understand patterns
4. Feature Engineering	Create better input variables
5. Model Selection	Pick the best algorithm
6. Hyperparameter Tuning	Adjust settings for better performance
7. Model Training and Evaluation	Train model and test accuracy
8. Model Deployment	Put model into real use

1. **Data Ingestion**
   → The process of **gathering data** from various sources:
   - CSV files
   - SQL databases
   - APIs
   - Real-time streams
2. **Data Preprocessing**
   → **Cleaning the raw data** to make it usable, including:
   - Handling missing values
   - Removing duplicates
   - Converting data types
   - Normalizing values
   - Fixing errors or inconsistencies

3. **Data Exploration**

   → Also known as **Exploratory Data Analysis (EDA)**. We analyze data to understand patterns, correlations, and potential issues, using:

   - Charts
   - Summaries
   - Visualizations

4. **Feature Engineering**
   → Creating or modifying **input variables (features)** to help the model learn better. This step transforms raw data into more meaningful representations. For example:

   - Converting `"date of birth"` into `"age"` to make it a usable number
   - Creating `"BMI"` from `"height"` and `"weight"`
   - Encoding categories (e.g., turning `"Male"` / `"Female"` into 0 and 1)
5. **Model Selection**
   → Choosing the **appropriate algorithm** based on the task (e.g., regression, classification, clustering). The choice depends on:

   - Data type
   - Project goal
   - Need for interpretability
6. **Hyperparameter Tuning**
   → Adjusting the model's settings (like tree depth, number of clusters, or learning rate) to improve performance.
   Good hyperparameters can greatly boost model accuracy and reduce overfitting.

   Common techniques include:

   - Grid Search: Tries all possible combinations of hyperparameter values
   - Random Search: Tries random combinations for faster results on large spaces

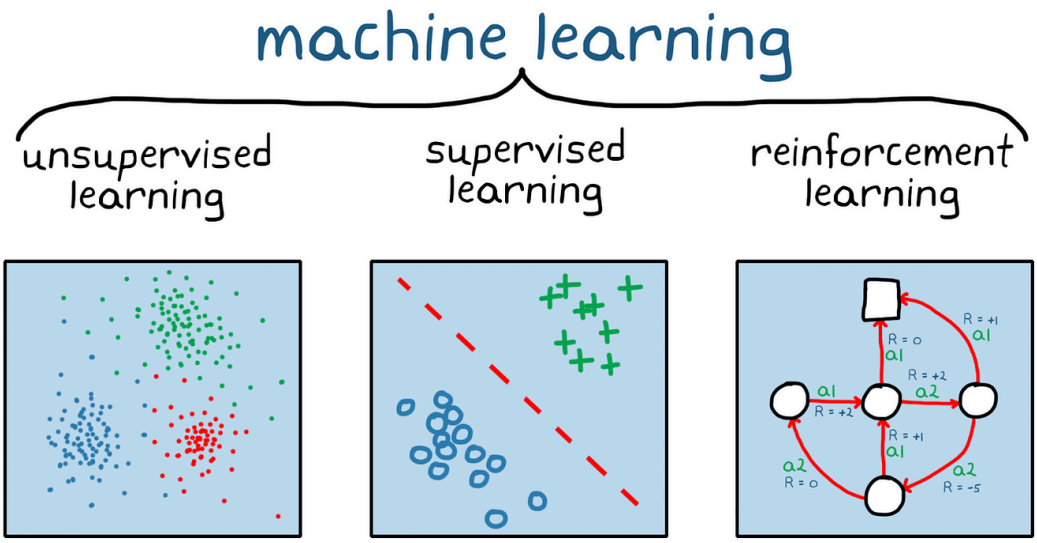
   These methods help find the best settings based on validation performance (e.g., accuracy or RMSE)

7. **Model Training and Evaluation**
   → **Training** the model on a training dataset and then **evaluating** it on a test dataset. We measure how well it performs using metrics like:

   - Accuracy
   - Precision & Recall
   - RMSE (Root Mean Square Error = how far your predictions are from the real values), etc.
8. **Model Deployment**
   → Putting the trained model into **real-world usage**, such as:

   - Integrating into a web or mobile app
   - Embedding into backend systems
   - Making real-time predictions on new, unseen data

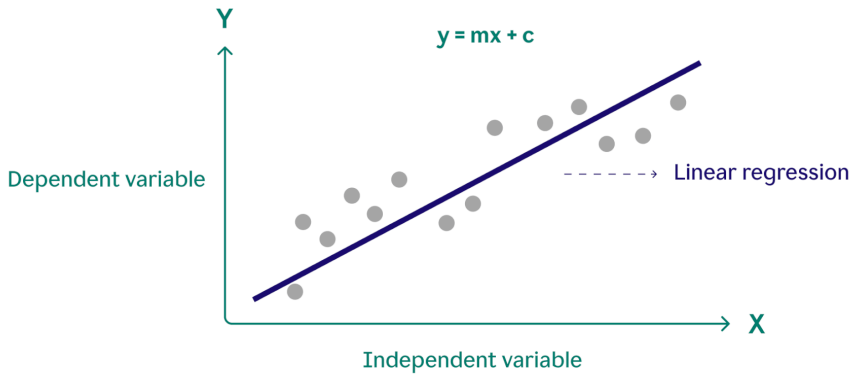
4. Types of Machine Learning



	Unsupervised Learning	Supervised Learning	Reinforcement Learning
Definition	Finding patterns in data without labeled examples	Learning with labeled examples (we know the correct answers)	Learning through trial and error with rewards/penalties
Examples	- Customer segmentation - Market basket analysis - Anomaly detection	- Email spam detection (spam/not spam) - House price prediction - Image recognition	- Game playing (chess, Go) - Robot navigation - Trading strategies
Common Algorithms	- K-Means Clustering - Hierarchical Clustering - Principal Component Analysis (PCA)	- Linear Regression - Decision Trees - Random Forest - Support Vector Machines	- Q-learning - Deep Q-Networks (DQN) - Policy Gradient Methods

5. Common Algorithms

1. Linear Regression

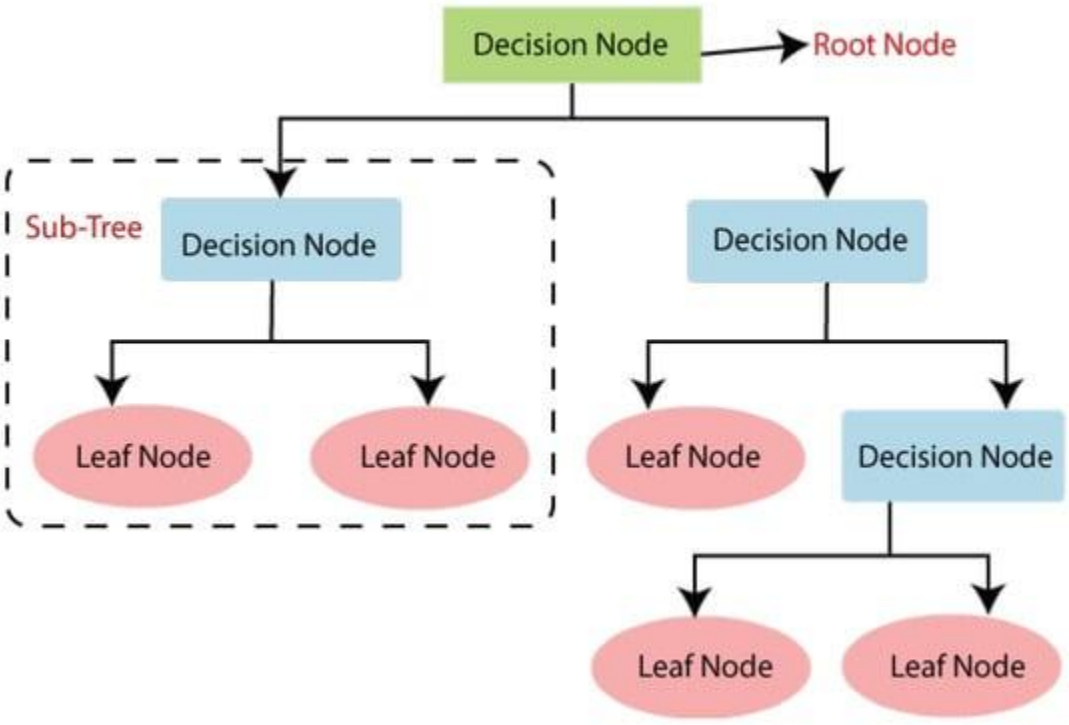


Definition: A model that predicts a continuous value based on the relationship between input features and the target.

Use Case: Predicting prices, trends (e.g., house price prediction).

Strengths	Weaknesses
Simple and easy to interpret	Struggles with non-linear patterns
Works well when the relationship is linear	Sensitive to outliers

2. Decision Trees

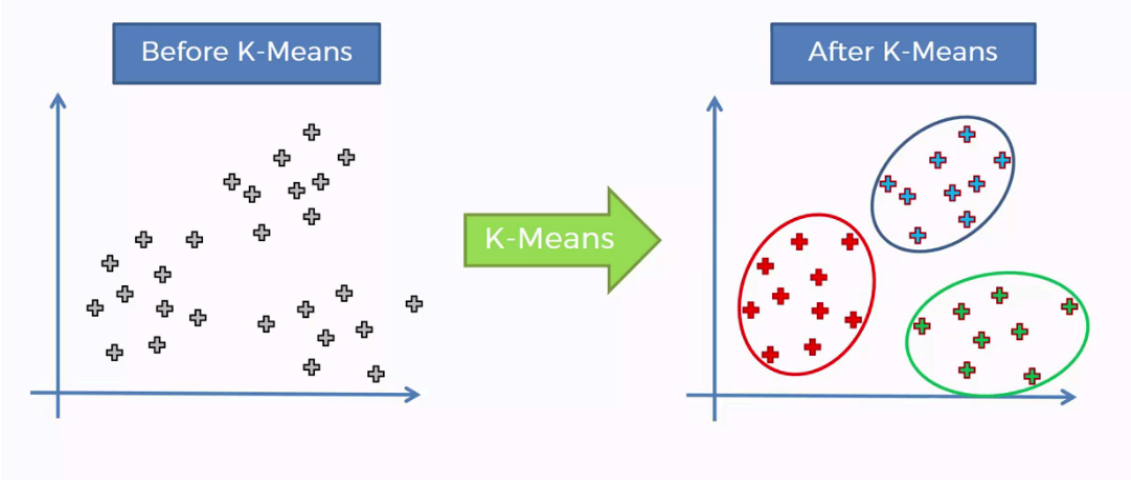


Definition: A tree-based model that splits the data based on feature values to make decisions.

Use Case: Classification problems like determining whether an email is spam or not.

Strengths	Weaknesses
Easy to understand and visualize	Prone to overfitting
Can handle both numerical and categorical data	Small changes in data can lead to a completely different tree
No need for feature scaling	

3. K-Means Clustering



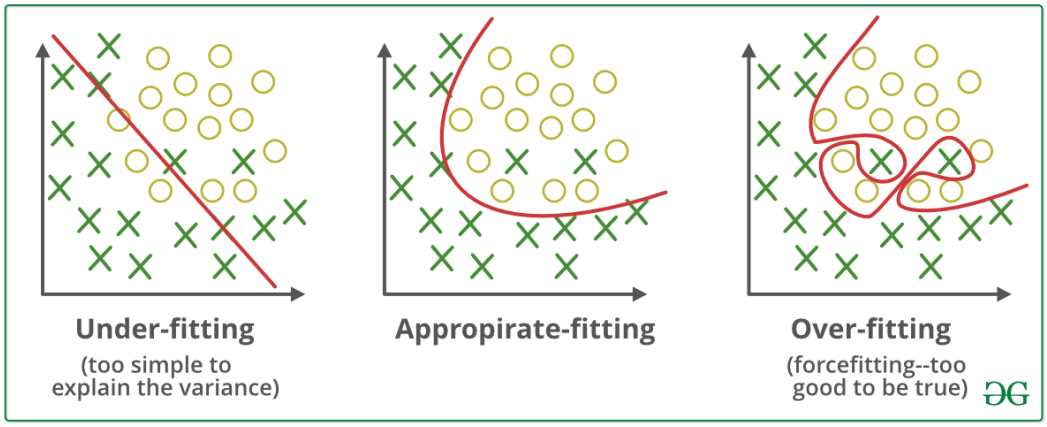
Definition: An unsupervised algorithm that groups data into K clusters based on similarity.

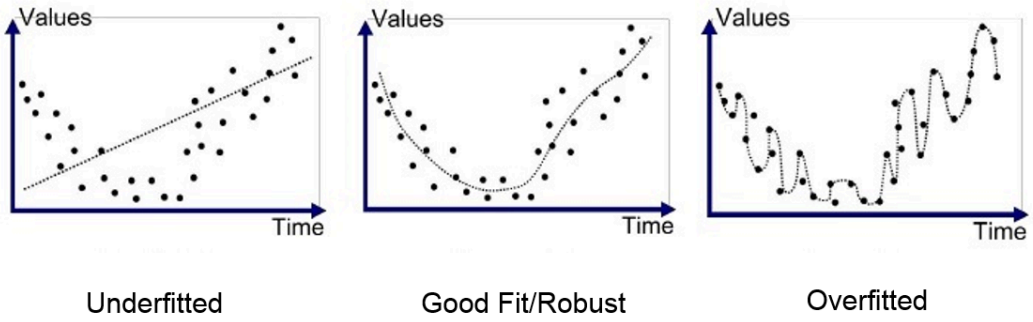
Use Case: Customer segmentation, grouping similar users or products.

Strengths	Weaknesses
Fast and efficient for large datasets	Requires choosing K (number of clusters) manually
Easy to understand and implement	Doesn't work well with non-spherical clusters or different cluster sizes
	Sensitive to outliers

6. Additionals

1. Overfitting vs Underfitting





	Underfitting (Too Simple)	Overfitting (Too Complex)	Good Fit (Just Right)
What it is	Model is too basic to capture real patterns	Model memorizes training data instead of learning general patterns	Model captures the true underlying patterns
Example	Using a straight line to predict house prices when the relationship is curved	A student who memorizes answers but can't solve new problems	A student who understands the concepts and can apply them in new situations
Signs	Poor performance on both training and test data	Great performance on training data, terrible on new data	Good performance on both training and test data
Symptoms	High bias, low variance	Low bias, high variance	Balanced bias and variance
Solution	Use more complex models or add more features	Use simpler models, get more data, or apply	Maintain a good balance between model complexity

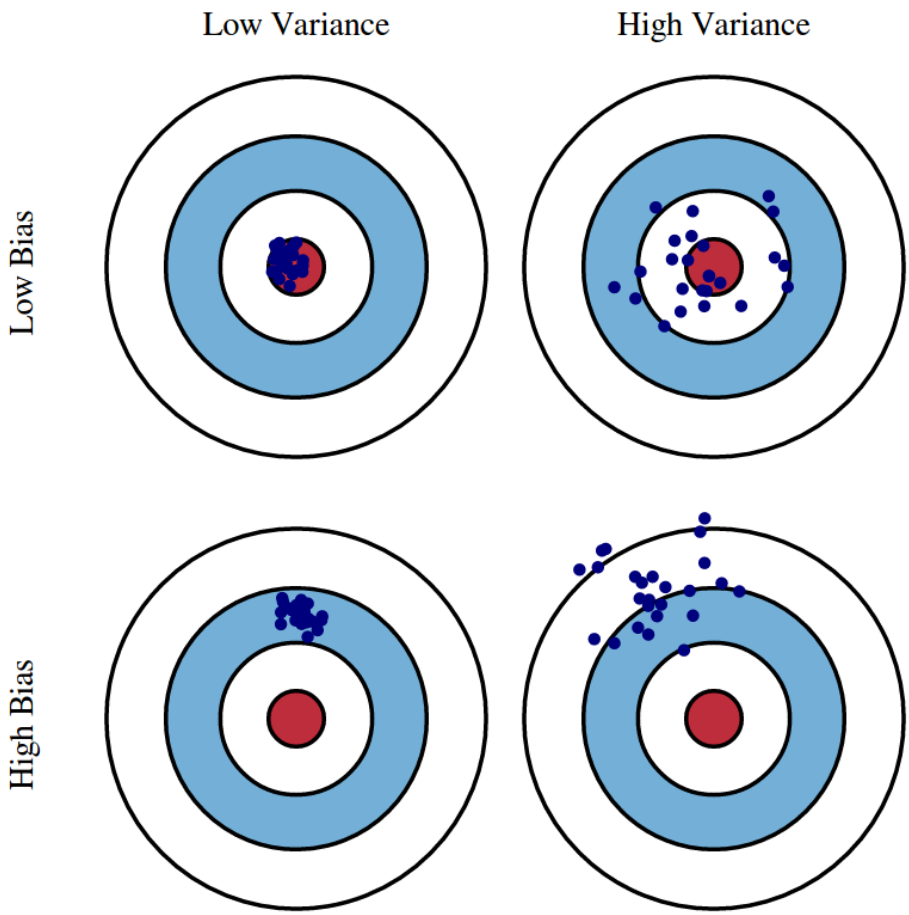
How to Detect:

```
# Training accuracy: 95%
# Test accuracy: 96%          ← Good fit!

# Training accuracy: 60%
# Test accuracy: 58%          ← Underfitting

# Training accuracy: 99%
# Test accuracy: 65%          ← Overfitting
```

2. Bias-Variance Tradeoff



- **Bias**: Error due to overly simple assumptions (underfitting)
- **Variance**: Error due to too much complexity (overfitting)
- **Goal**: Find a balance where both are low

```
High Bias → Model is too simple → Underfitting
High Variance → Model is too complex → Overfitting
Just Right → Low bias and low variance → Good generalization
```

3. Tips to Increase Model Accuracy

- Clean your data properly
- Feature engineering (create meaningful input variables)
- Try different models and compare
- Use cross-validation

- Tune hyperparameters (e.g., learning rate, depth, number of trees)
- Add more high-quality data
- Use ensemble methods (e.g., Random Forest, XGBoost)
- Regularize your models (e.g., L1/L2 regularization)